Programação Orientada a Objetos – Engenharia de Telecomunicações

Associação entre classes

Modelagem de entidades e relacionamentos, e o princípio da responsabilidade única

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

INSTITUTO FEDERAL
Santa Catarina
Campus São José

Licenciamento

Slides licenciados sob Creative Commons “Atribuição 4.0 Internacional”

Modelando entidades e relacionamentos

Modelagem de Sistema de Informação I

Entidades e relacionamentos

- A modelagem de **entidades** e **relacionamentos** define a estrutura dos dados e as interações entre eles
- Indica quais informações são relevantes para o sistema, ou seja, quais informações serão armazenadas e manipuladas

Modelagem de Sistema de Informação II

Entidades e relacionamentos

- **Entidade** é um objeto do mundo real que é **distinguível de todos os outros objetos**
- Exemplos
  - `Aluno` "Juca"
  - `Livro` "O Senhor dos Anéis"
  - `Carro` "Fusca"

Modelagem de Sistema de Informação III

Entidades e relacionamentos

- **Relacionamento** é uma associação entre duas ou mais **entidades**
- Exemplos
  - `Aluno` **possui** `Matrícula` em `Curso`
  - `Livro` **escrito por** `Autor`
  - `Carro` **pertence a** `Pessoa`

Modelagem de Sistema de Informação IV

Entidades e relacionamentos

- **Atributos** são propriedades de uma **entidade** ou de um **relacionamento**
- Exemplos de atributos em entidades
  - Todo `Aluno` tem `nome`
  - Todo `Livro` tem `título`, `ISBN`
  - Todo `Carro` tem `modelo`, `ano`
- Exemplos de atributos em relacionamentos
  - O relacionamento `Aluno` **possui** `Matrícula` em `Curso` tem o atributo `data de matrícula`
  - O relacionamento `Livro` **escrito por** `Autor` tem o atributo `data de publicação`
  - O relacionamento `Carro` **pertence a** `Pessoa` tem o atributo `data de compra`

Modelagem de entidades e relacionamentos

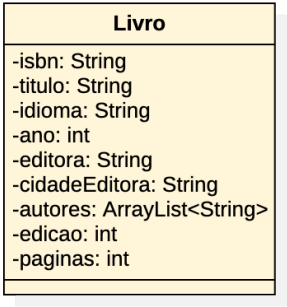
Linguagem orientada a objetos

- Em linguagens orientadas a objetos, as **entidades** são instâncias de **classes**, os **atributos** são **variáveis de instância** e os **relacionamentos** são **associações** entre classes
- A modelagem de entidades e relacionamentos pode ser representada por meio de diagramas de classes UML

> **Nota**
>
> *Frameworks* de Mapeamento Objeto-Relacional (ORM, *Object-Relational Mapping*) permitem mapear as classes do sistema para tabelas em um banco de dados relacional, facilitando a persistência dos dados

Modelando entidades e relacionamentos

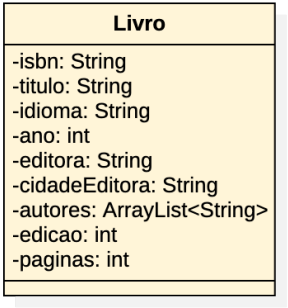
Sistema para gestão de livros



- A classe Livro deve representar uma entidade do mundo real, ou seja, um livro
- Os atributos são as informações que serão armazenadas sobre cada livro
- O livro não deve ter atributos que não sejam dele, ou seja, que sejam de outras entidades ou ainda de relacionamentos entre entidades
- Quais propriedades não são da entidade livro?

Modelando entidades e relacionamentos

Sistema para gestão de livros

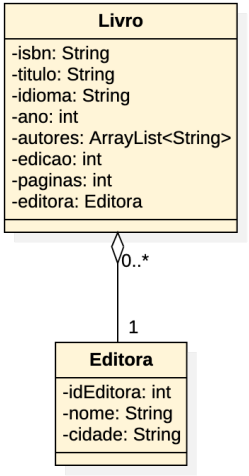


- Para um livro é importante saber o **nome da editora**
- A **cidade da editora** não é uma propriedade do livro, mas sim da editora
- Uma **editora** pode publicar vários livros

Um relacionamento entre as classes `Livro` e `Editora` para representar essa associação seria um modelo mais adequado do que colocar o nome da editora como um atributo do livro

Modelando entidades e relacionamentos

Sistema para gestão de livros

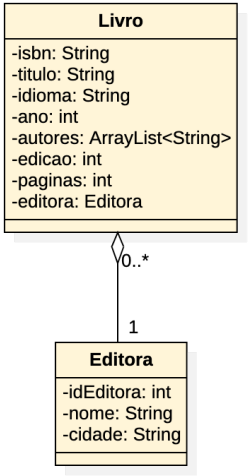


- Para um livro é importante saber o **nome da editora**
- A **cidade da editora** não é uma propriedade do livro, mas sim da editora
- Uma **editora** pode publicar vários livros

Um relacionamento entre as classes `Livro` e `Editora` para representar essa associação seria um modelo mais adequado do que colocar o nome da editora como um atributo do livro

Modelando entidades e relacionamentos

Sistema para gestão de livros



- Um `Livro` pode ter vários autores e um `Autor` pode ter escrito vários livros
- Sobre o Autor, é importante registrar mais informações além do nome, como por exemplo a data de nascimento, nacionalidade, etc

Um relacionamento entre as classes `Livro` e `Autor` para representar essa associação seria um modelo mais adequado

Modelando entidades e relacionamentos

Sistema para gestão de livros

- Um `Livro` pode ter vários autores e um `Autor` pode ter escrito vários livros
- Sobre o Autor, é importante registrar mais informações além do nome, como por exemplo a data de nascimento, nacionalidade, etc

Um relacionamento entre as classes `Livro` e `Autor` para representar essa associação seria um modelo mais adequado

Modelando entidades e relacionamentos

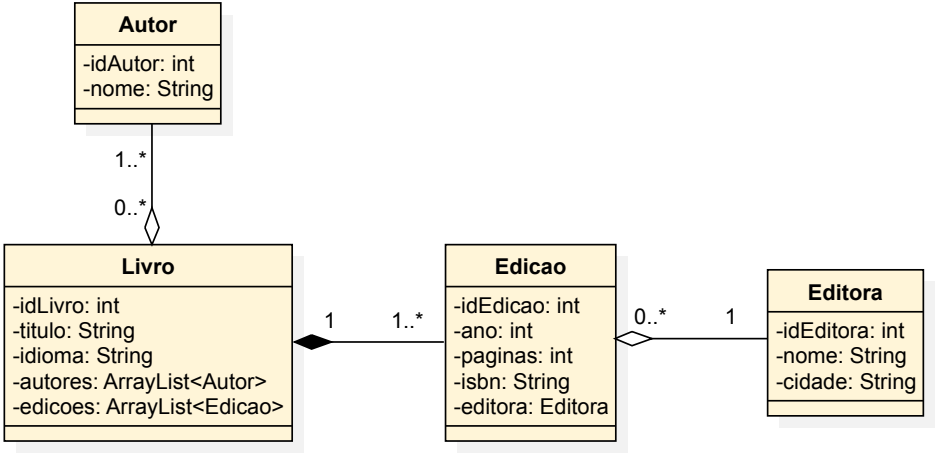
Sistema para gestão de livros

- Cada livro pode ter várias edições
- Cada edição tem um ISBN, um número de páginas e um ano de publicação
- Cada edição pode ser publicada por uma editora diferente

Vamos praticar!

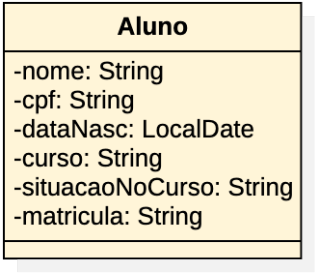
Ajuste o diagrama para representar as edições dos livros, considerando os atributos de cada edição e a editora responsável por cada edição

Um modelo possível para o sistema de gestão de livros



Modelando entidades e relacionamentos

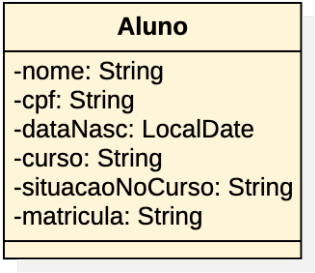
Sistema acadêmico



- Existem propriedades que são de outras entidades ou ainda de relacionamentos entre entidades?

Modelando entidades e relacionamentos

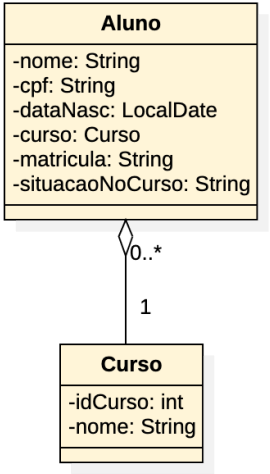
Sistema acadêmico



- `curso`, que guarda o nome do curso, não é uma propriedade de `Aluno`, mas sim de `Curso`
- Um `Curso` pode ter vários alunos

Modelando entidades e relacionamentos

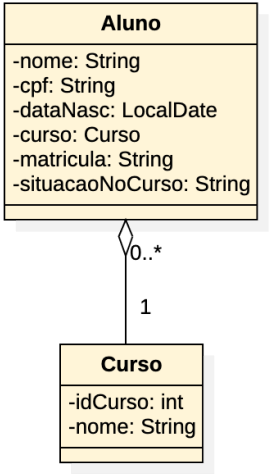
Sistema acadêmico



- `curso`, que guarda o nome do curso, não é uma propriedade de `Aluno`, mas sim de `Curso`
- Um `Curso` pode ter vários alunos

Modelando entidades e relacionamentos

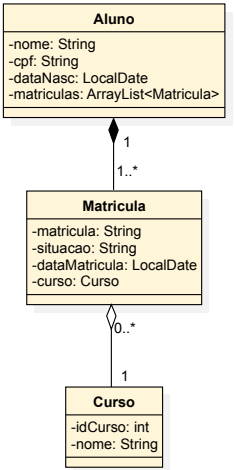
Sistema acadêmico



- É importante registrar a data da matrícula do aluno no curso
- Um aluno pode fazer vários cursos
- A situação do aluno no curso não é uma propriedade do aluno, mas sim de sua matrícula

Modelando entidades e relacionamentos

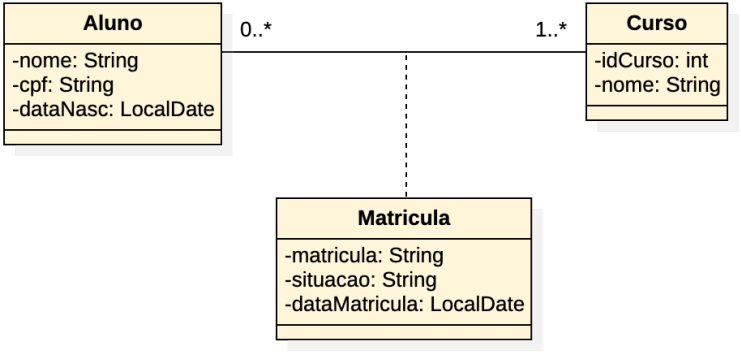
Sistema acadêmico



- É importante registrar a data da matrícula do aluno no curso
- Um aluno pode fazer vários cursos
- A situação do aluno no curso não é uma propriedade do aluno, mas sim de sua matrícula

Classe de associação

- A associação entre classes pode ser redefinida por uma nova classe a qual terá atributos próprios
- Adequada quando os atributos não pertecem a nenhuma das classes envolvidas na associação, mas à associação em si



Princípio da responsabilidade única

Princípio da responsabilidade única

***Single Responsibility Principle* (SRP)**[1]

> *"Ao escrever um módulo de software, você deve se lembrar de que ele deve ter uma e apenas uma razão para mudar, isto é, uma responsabilidade. Essas mudanças só podem ser solicitadas por uma única pessoa (Stakeholder)"* – **Robert C. Martin, *Uncle Bob***

- **Cada classe deve ser responsável por uma pequena parte** da funcionalidade de um software
- A **responsabilidade** deve **estar completamente encapsulada** dentro da classe

[1] https://blog.cleancoder.com/uncle-bob/2014/05/08/SingleReponsibilityPrinciple

Separação de preocupações

***Separation of Concerns* (SoC)**

- **Dividir** um sistema em **partes** que **cada uma** é **responsável por uma preocupação** diferente
- **Cada parte** é **desenvolvida, testada e mantida** separadamente
- Isso **facilita** a **manutenção**, **reutilização** e **evolução** do sistema

Exemplos

- Um sistema acadêmico pode ser dividido em módulos para gerenciar alunos, disciplinas, livros, etc
- Um sistema *web* pode ser dividido em módulos para gerenciar a interface com o usuário, a lógica de negócios e o armazenamento de dados

Princípio da responsabilidade única

Exemplo

Sistema para gestão de Agenda telefônica

Requisitos e regra de negócio

- O sistema deve permitir ao usuário gerenciar uma agenda telefônica, onde é possível armazenar informações de contatos pessoais e profissionais
  - Adicionar, Remover, Atualizar, Listar dados de um contato, Listar todos contatos
- Todo contato deve possuir:
  - Nome, Sobrenome, Data de nascimento, Telefone(s) e email(s)
- Todo telefone ou email deve possuir um rótulo de identificação (p. ex. celular, comercial, pessoal) e um valor (p. ex. pessoal: juca@example.com)
  - Não deve ser permitido adicionar um email que não seja válido
  - Os telefones devem ser exibidos formatados, no formato internacional: Ex: +55048998761234 → +55 (48) 9 9876-1234
- Ao listar os contatos, deve-se exibir o nome completo, data de nascimento, telefone(s) e email(s) de cada contato
- Os telefones e emails devem ser exibidos com o rótulo de identificação

Sistema para gestão de Agenda telefônica

Seguindo o princípio da responsabilidade única, como você modelaria o sistema?

Vamos praticar!

Faça um diagrama de classes UML com **somente os nomes das classes** que serão necessárias, **bem como suas associações**

Sistema para gestão de Agenda telefônica

Quantas classes serão necessárias?

Sistema para gestão de Agenda telefônica

Quantas classes serão necessárias?

- `App` e `Contato`?

Sistema para gestão de Agenda telefônica

Quantas classes serão necessárias?

- `App` e `Contato`?
  - `App` está assumindo a lógica de gerenciar contatos
  - A classe `App` deve ser responsável somente pela interface com o usuário, não deve conter lógica de negócio

Sistema para gestão de Agenda telefônica

Quantas classes serão necessárias?

- `App` e `Contato`?
  - `App` está assumindo a lógica de gerenciar contatos
  - A classe `App` deve ser responsável somente pela interface com o usuário, não deve conter lógica de negócio
- `App`, `Agenda` e `Contato`?

Sistema para gestão de Agenda telefônica

Quantas classes serão necessárias?

- `App` e `Contato`?
  - `App` está assumindo a lógica de gerenciar contatos
  - A classe `App` deve ser responsável somente pela interface com o usuário, não deve conter lógica de negócio
- `App`, `Agenda` e `Contato`?
  - Quem será responsável por garantir que os emails são válidos?
  - A classe `Contato` deve ser responsável somente por armazenar os dados do contato, não deve conter lógica para validar emails

Sistema para gestão de Agenda telefônica

Quantas classes serão necessárias?

- `App` e `Contato`?
  - `App` está assumindo a lógica de gerenciar contatos
  - A classe `App` deve ser responsável somente pela interface com o usuário, não deve conter lógica de negócio
- `App`, `Agenda` e `Contato`?
  - Quem será responsável por garantir que os emails são válidos?
  - A classe `Contato` deve ser responsável somente por armazenar os dados do contato, não deve conter lógica para validar emails
- `App`, `Agenda`, `Contato` e `ColecaoEmail`?

Sistema para gestão de Agenda telefônica

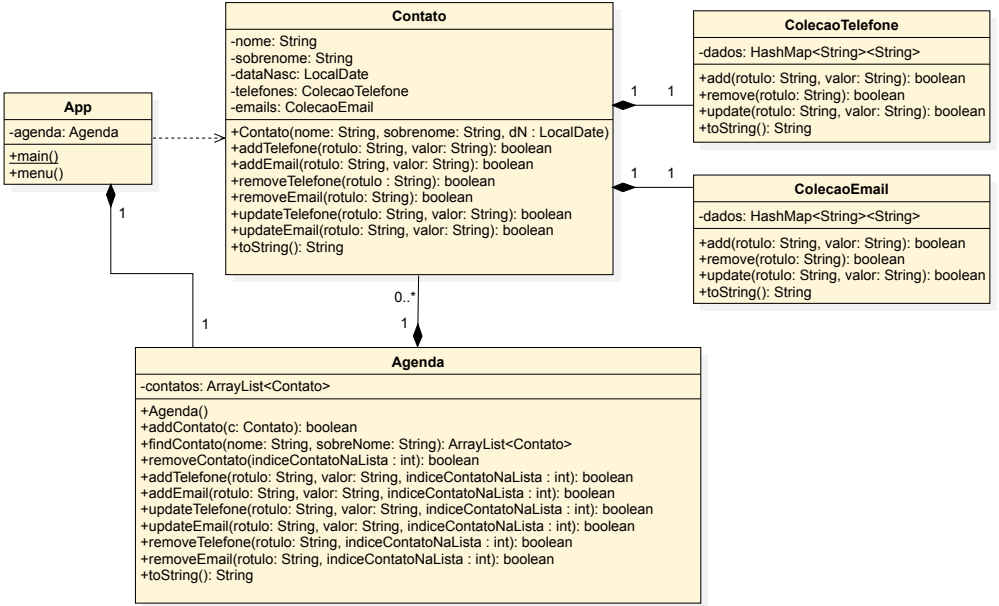
Quantas classes serão necessárias?

- `App` e `Contato`?
  - `App` está assumindo a lógica de gerenciar contatos
  - A classe `App` deve ser responsável somente pela interface com o usuário, não deve conter lógica de negócio
- `App`, `Agenda` e `Contato`?
  - Quem será responsável por garantir que os emails são válidos?
  - A classe `Contato` deve ser responsável somente por armazenar os dados do contato, não deve conter lógica para validar emails
- `App`, `Agenda`, `Contato` e `ColecaoEmail`?
  - Quem será responsável por formatar os telefones?

Sistema para gestão de Agenda telefônica

Quantas classes serão necessárias?

- `App` e `Contato`?
  - `App` está assumindo a lógica de gerenciar contatos
  - A classe `App` deve ser responsável somente pela interface com o usuário, não deve conter lógica de negócio
- `App`, `Agenda` e `Contato`?
  - Quem será responsável por garantir que os emails são válidos?
  - A classe `Contato` deve ser responsável somente por armazenar os dados do contato, não deve conter lógica para validar emails
- `App`, `Agenda`, `Contato` e `ColecaoEmail`?
  - Quem será responsável por formatar os telefones?
- `App, Agenda, Contato, ColecaoEmail e ColecaoTelefone`


Contato
-nome: String
-sobrenome: String
-dataNasc: LocalDate
-telefones: ColecaoTelefone
-emails: ColecaoEmail
+Contato(nome: String, sobrenome: String, dN : LocalDate)
+addTelefone(rotulo: String, valor: String): boolean
+addEmail(rotulo: String, valor: String): boolean
+removeTelefone(rotulo : String): boolean
+removeEmail(rotulo: String): boolean
+updateTelefone(rotulo: String, valor: String): boolean
+updateEmail(rotulo: String, valor: String): boolean
+toString(): String
ColecaoTelefone
-dados: HashMap<String><String>
+add(rotulo: String, valor: String): boolean
+remove(rotulo: String): boolean
+update(rotulo: String, valor: String): boolean
+toString(): String
ColecaoEmail
-dados: HashMap<String><String>
+add(rotulo: String, valor: String): boolean
+remove(rotulo: String): boolean
+update(rotulo: String, valor: String): boolean
+toString(): String
App
-agenda: Agenda
+main()
+menu()
1
1
1
1
1
1
0..*
1
Agenda
-contatos: ArrayList<Contato>
+Agenda()
+addContato(c: Contato): boolean
+findContato(nome: String, sobreNome: String): ArrayList<Contato>
+removeContato(indiceContatoNaLista : int): boolean
+addTelefone(rotulo: String, valor: String, indiceContatoNaLista : int): boolean
+addEmail(rotulo: String, valor: String, indiceContatoNaLista : int): boolean
+updateTelefone(rotulo: String, valor: String, indiceContatoNaLista : int): boolean
+updateEmail(rotulo: String, valor: String, indiceContatoNaLista : int): boolean
+removeTelefone(rotulo: String, indiceContatoNaLista : int): boolean
+removeEmail(rotulo: String, indiceContatoNaLista : int): boolean
+toString(): String

Material de apoio

Classes `java.time.LocalDate` e `java.time.LocalDateTime`

- `java.time.LocalDate`

```java
import java.time.Month;
//.....
// Dia de hoje
LocalDate hoje = LocalDate.now();
// Criando objeto com uma data especifica
LocalDate dataEspecifica = LocalDate.of(2020, Month.JANUARY, 10);
```

- `java.time.LocalDateTime`

```java
import java.time.format.DateTimeFormatter;
//.....
String dHF;

LocalDateTime dataHora = LocalDateTime.of(2019, 04, 22, 13, 30, 00);
dhF = dataHora.format(DateTimeFormatter.ofPattern("yyyyMMdd'T'HHmmss"));
System.out.printf(dHF); // Será impresso: 20190422T133000

// para obter a data e hora no momento que essa linha foi executada
LocalDateTime agora = LocalDateTime.now();
```

Classes `LocalDate`[2] e `DateTimeFormatter`[3]

Para converter String para formato de data

```java
Scanner teclado = new Scanner(System.in);

// O usuário informa a data no formato dd/mm/aaaa
System.out.print("Entre com a data (dd/mm/aaaa): ");
String dataString = teclado.nextLine();

// Formato da data - d representa dia, M representa mês e y representa ano
DateTimeFormatter formato = DateTimeFormatter.ofPattern("dd/MM/yyyy");

// Converte a String para LocalDate, considerando o formato informado na String
LocalDate data = LocalDate.parse(dataString, formato);

// Exibe a data no formato padrão que é yyyy-MM-dd
System.out.println("Data: " + data);

// Exibe a data no formato informado pelo objeto formato
System.out.println("Data: " + data.format(formato));
```

[2] https://docs.oracle.com/javase/tutorial/datetime/iso/index.html

[3] https://docs.oracle.com/javase/8/docs/api/java/time/format/DateTimeFormatter.html

Expressão regular (*regex*)

Metacaractere	Descrição
`.`	Casa com qualquer caractere
`[]`	Lista de caracteres. Ex: `n[aã]o` casa com *não* e *nao*. É possível definir intervalos [0-2] casa com 0, 1 ou 2
`^`	Casa com o começo da cadeia de caracteres
`$`	Casa com o final da cadeia de caracteres
`*`	Casa o elemento precedente zero ou mais vezes. Ex: `ab*c` casa com ac, abc, abbbbc
`+`	Casa o elemento precedente um ou mais vezes
`?`	Torna o elemento precedente opcional
`!`	Negação. Ex: `n[!a]` casa com n, mas não com na, nem não
`\`	Para escapar outros metacaracteres.
`{n,m}`	De n até m. Ex: `2,` pelo menos 2 caracteres
`()`	Grupo. Ex: `(www\.)?ifsc.edu.br` casa com www.ifsc.edu.br e ifsc.edu.br
`\w`	Corresponde a `[a-zA-Z_0-9]`

Expressão regular (*regex*)

Exemplos em Java

```java
// Para garantir que somente números sejam digitados
String eR = "^[0-9]+$";
String numero = "1234567890";
System.out.println("Número válido? " + numero.matches(eR));

// Para garantir que somente letras sejam digitadas
String eR = "^[a-zA-Z]+$";
String letras = "abcde";
System.out.println("Letras válidas? " + letras.matches(eR));

// Para garantir que somente letras e números sejam digitados
String eR = "^[a-zA-Z0-9]+$";
String alfanumerico = "abc123";
System.out.println("Alfanumérico válido? " + alfanumerico.matches(eR));

// Para não permitir caracteres repetidos e que tenha pelo menos 8 caracteres
String eR = "^(?!.*(.)\\1{2,})[a-zA-Z0-9]{8,}$";
String senha = "abc12345";
System.out.println("Senha válida? " + senha.matches(eR));
```

Expressão regular (*regex*)

Exemplo de validação de email

```
String eR = "^[\\w-\\+]+(\\.[\\w]+)*@[\\w-]+(\\.[\\w]+)*(\\.[a-z]{2,})$";

String email = "meu.email@dominio.com.br";

System.out.println("Email válido? " + email.matches(eR));
```

- https://docs.oracle.com/javase/tutorial/essential/regex/
- https://docs.oracle.com/javase/8/docs/api/java/util/regex/Pattern.html

Classe MaskFormatter

- A classe `MaskFormatter` é usada para formatar e editar Strings
- A máscara indica quais são os caracteres válidos que podem estar contidos na String

```
MaskFormatter mask = new MaskFormatter("(##) #####-####");
```

Símbolo	Descrição
#	Qualquer número
U	Qualquer caractere e todos serão convertidos para maiúsculo
L	Qualquer caractere e todos serão convertidos para minúsculo
A	Qualquer caractere ou número
?	Qualquer caractere
*	Qualquer coisa
H	Qualquer hexadecimal (0-9, a-f ou A-F)

Exemplo com classe MaskFormatter

```java
public String formata(String mascara, String valor){
    MaskFormatter mask = null;
    String resultado = "";

    try {
        mask = new MaskFormatter(mascara);
        mask.setValueContainsLiteralCharacters(false);
        mask.setPlaceholderCharacter('_');
        resultado = mask.valueToString(valor);
    } catch (ParseException e) {
        e.printStackTrace();
    }

    return resultado;
}

public static void main(String[] args) {
    App p = new App();
    System.out.println(p.formata("(##) #####-####", "48998765432"));
}
```

Sugestões de leitura

- Robert C. Martin
  Código limpo: Habilidades práticas do Agile Software
  1a. Edição - Editora Alta Books, 2009
  https://app.minhabiblioteca.com.br/reader/books/9788550816043
- Robert C. Martin
  Arquitetura Limpa: O guia do artesão para estrutura e design de software
  1a. Edição - Editora Alta Books, 2019
  https://app.minhabiblioteca.com.br/reader/books/9788550808161

Aula baseada em

Craig Larman

Utilizando UML e padrões

3a. Edição - Editora Bookman, 2007